CSC

ICT Solutions for Brilliant Minds

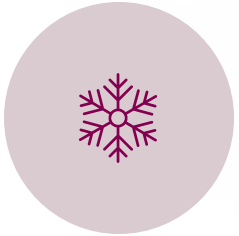
# HPCG for SCC

Background, installing and running

# This lecture:

WHAT IS HPCG?

ARITHMETIC INTENSITY

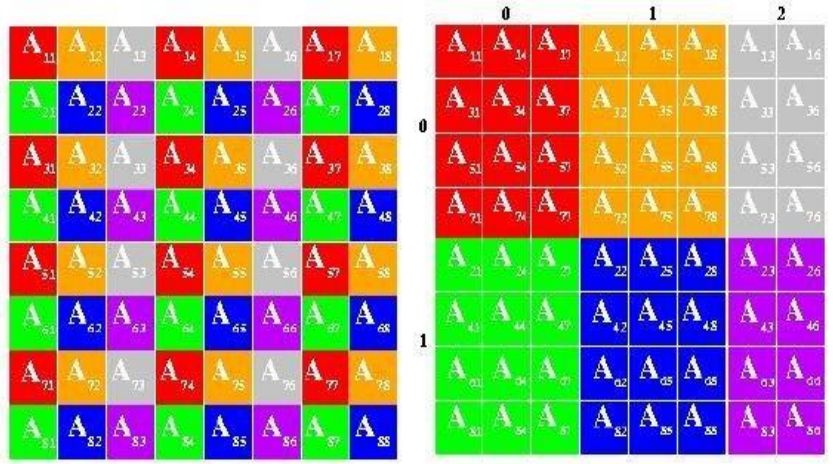


INSTALLING HPCG ON LUMI

RUNNING AND TUNING HPCG

# Reminder: HPL

- The "main" Top500 list is based on results from HPL
- Solves a (random) dense linear system of (Ax = b) in double precision
- The algorithm of the software is based in matrix multiplication
  - Results get close to the theoretical maximum performance of the hardware
- Great scaling on even very large clusters
- The algorithm can utilize GPUs well

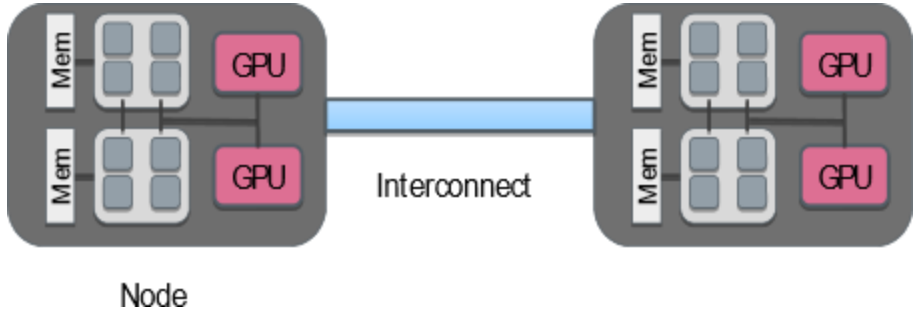


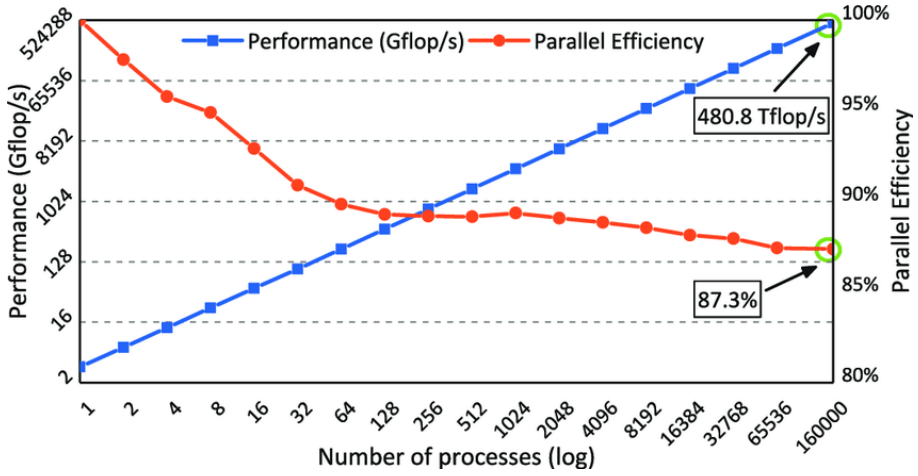

$$\begin{bmatrix} 1 & 3 & 1 \\ 2 & 2 & 2 \\ 3 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 5 \\ 6 \\ 5 \end{bmatrix}$$

A x b

# HPCG - High-Performance Conjugate Gradients

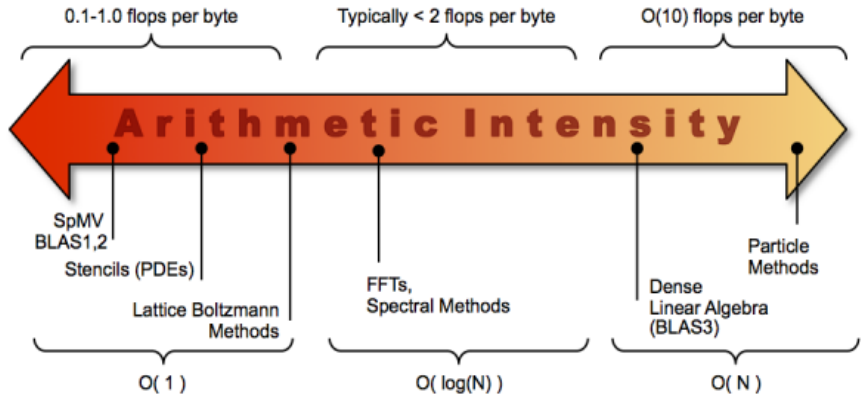
- Computes a (sparse) system of linear equations
  - "Preconditioned conjugate gradient iterations"
  - Tests data access patterns that are more often represented in real-world applications
- Solving the problem puts more stress on the memory/interconnect of the system
- Used alongside HPL to complement it
  - Is well suited for GPUs
  - Results scale well even on larger runs



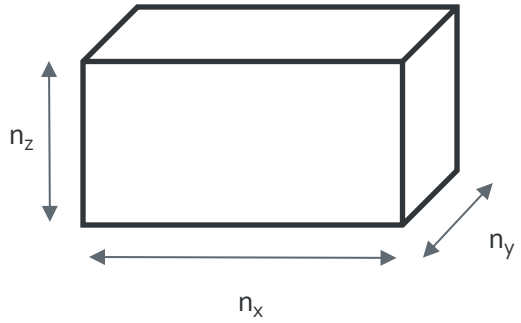
# HPL vs. HPCG

- HPCG was developed a bit later to complement HPL
- HPCG gives a very low FLOPs result compared to HPL
  - The algorithm has a lower arithmetic intensity
  - Memory access speeds create a bottleneck
- For LUMI: 1.2% compared to HPL
- With HPL and HPCG you have benchmarks that represent two extreme ends of a spectrum
  - Real-world applications usually fall somewhere between these two
  - Computation-bound vs. Memory-bound



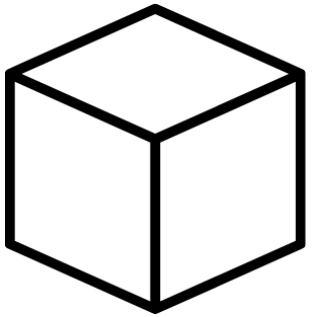
# HPCG - Input parameters

- The size of the matrix that will be solved ($n_x$, $n_y$, $n_z$)
  - Local for each MPI process
- Running time (in seconds)
- The problem size should be large enough to be an officially valid result
  - Typically, around 25% of the local memory in the machine
  - Running time of over 1800s



# HPCG - Tuning

- The size of the matrix
  - Similar to HPL -> up to 80-90% of max memory
  - Large problem sizes can take tens of minutes to initialize
  - In an ideal case nx = ny = nz
- Make sure that the GPUs in the system are working a 100%
  - The "rocm-smi" command
  - Rules out issues with MPI/load imbalance and thermal throttling



# Brief side note - Installing things on LUMI

- LUMI is a Cray machine
  - Employs a proprietary Cray software stack
  - Compilers and the underlying MPI in each program needs to use this Cray stack correctly
- Cray compiler wrappers (use these when compiling)
  - C: cc
  - CPP: CC
  - Fortran: FTN
- Cray MPI
  - The cray-mpich -module
  - Don't try to install your own MPI locally, it will not work on the system correctly
  - Add the environment variable "MPICH_GPU_SUPPORT_ENABLED=1" when running

# HPCG - Installing and running

- Instructions in this week's exercise folder
- We will use rocHPCG, which is a specifically tuned version for AMD GPUs
- Install rocHPCG with the following in mind:
  - Do not install any dependencies!
  - Have the "LUMI/23.09", "partition/G" and "rocm" modules loaded
  - Use the system MPI (module show cray-mpich)
  - Include the "--gpu-aware-mpi=true" compilation flag
- We are working on LUMI; we need to use LUMI's Cray compiler wrappers
  - export CXX=CC
  - export HIPCXX=CC

CSC


facebook.com/CSCfi


twitter.com/CSCfi


linkedin.com/company/csc---it-center-for-science


github.com/CSCfi